

Jakarta Messaging

Dr Olalekan Samuel Ogunleye
School of Computing and Mathematical Science
University of Mpumalanga

Objectives of this Session

By the end of this session, you should be able to:

- Understand the Jakarta Messaging

- Understand how to Create a Producer to Queue

- Understand how Create a Consumer to Queue

- Understand how to Create a Producer to Topic

- Understand how Create a Consumer to Topic

- Understand how Create a Consumer using message-driven bean

Understanding the Jakarta Messaging

When we develop an enterprise application, in general, this application communicates with the other applications.

Therefore, many times, the enterprise application needs to send the messages to each other in a loosely coupled and asynchronous way.

With this, the enterprise application should have some known way to send the messages between these applications.

The applications that will make a communication should know the message pattern and how to send and receive the messages.

Understanding the Jakarta Messaging

The pattern is very important to integrate the applications in a good way because if all the applications know the message's patterns, all the applications will know how to send and receive the messages.

With this, the Jakarta EE has the Jakarta Messaging to provide a solution based on the specification and to treat the integration between Jakarta applications using the messaging approach.

Understanding the Jakarta Messaging

Jakarta Messaging defines the producer that produces the message and sends it to a queue or topic.

Also, it defines the consumer, that consumes the message from the queue or topic.

The producer and consumer can be in different applications or in the same application, although, for the most part, they are in different applications.

The Jakarta Messaging has a good integration with Jakarta Enterprise Bean and Jakarta Transaction API.

Understanding the Jakarta Messaging

It means that Jakarta Messaging can participate in the transaction and can take advantage of the container-managed transaction from Jakarta Messaging.

It also means that if the Jakarta Messaging is being used inside the session bean of Jakarta Enterprise Bean, the rollback of the sending message can be called in a case of any unchecked exception, without the action of the developer.

Even if the message was sent and the transaction was not committed, the message can be rolled back.

Understanding the Jakarta Messaging - Queue

The queue approach allows the application to send a message to a queue and the other client gets this message from the queue.

It is called a point-to-point (PTP) messaging style as well.

In case if a client gets the message from the queue, another client will not be able to get the same message, but will get the next.

The queue retains a message until this message is consumed or when it expires. Therefore, if we have many clients getting the message from the queue, it does not guarantee which client will get the message.

Understanding the Jakarta Messaging - Queue

Each message is consumed by only one client. The queue approach should be used when the message should be read and processed by only one client.

Understanding the Jakarta Messaging - Topic

In the topic approach, which is also known as the Publish/Subscribe messaging style, the message is read and processed by all the clients subscribed to the topic.

As the name shows, the topic works with a publisher that sends the message to the topic, and the subscribers read the message from the topic and process it.

The main difference between a queue and a topic is that the topic can have multiple consumers and all the consumers subscribed will receive the messages.

By default, the consumer just receives the messages sent after subscribing, but Jakarta Messaging provides a way to allow the consumer to consume messages sent to the topic before subscribing to this topic.

Understanding the Jakarta Messaging - Topic

To do that, use the PERSISTENT delivery mode for the publishers and a durable subscription for the subscribers.

Note that the subscription and the consumer are not the same thing.

The topic has the subscription that receives all the messages sent to the topic, according to the configuration, and the consumer that is associated with the subscription.

One subscription can have multiple consumers, and this is called a shared subscription, or you can have a subscription with just one consumer, that is called an unshared subscription. Throughout this session, we will explore more of these subscriptions.

Understanding the Jakarta Messaging - Topic

In Jakarta Messaging, the messages can be consumed synchronously or asynchronously.

In a synchronous way, the consumer calls the receive method to get a message and it will block until a message arrives or it takes a time out. In an asynchronous way, a message listener, or a Message-Driven Bean (MDB) is registered to the listener of the messages.

Understanding the Jakarta Messaging - Topic

The Jakarta Messaging has two parts - the connection factory and the destinations.

The connection factory encapsulates the logic and its configurations to create the connections.

The connection factory is an instance of the **ConnectionFactory**, **QueueConnectionFactory**, or **TopicConnectionFactory** interface.

The ConnectionFactory can be injected using CDI, shown as follows:

Understanding the Jakarta Messaging - Topic

```
@Resource(lookup = "java:jboss/ Default MSConnectionFactory")  
private static ConnectionFactory connectionFactory;
```

Understanding the Jakarta Messaging - Topic

destination is an object that represents the target of the message that will be sent.

In case of the queue, the object will be of Queue and in case of a topic, the object will be of Topic.

The destination can be injected using CDI as well, shown as follows:

```
@Resource(lookup = "java:/jms/queue/MyQueue")
```

```
private static Queue queue;
```

```
@Resource(lookup = "java:/jms/topic/MyTopic")
```

```
private static Topic topic;
```

Understanding the Jakarta Messaging - Message

The message is represented by an object that can be of these types - Message, BytesMessages, MapMessage, ObjectMessage, StreamMessage, and TextMessage.

These messages consist of a header, properties, and a body.

The header message contains information about the identity and route, properties are additional fields that can be added to the message, and the body contains the content that the client sent to the provider.

When a client wants to send a message to a queue/topic, it creates a producer and the message.

Understanding the Jakarta Messaging - Message

Then the message is sent using the producer. Therefore, the client that wants to consume the message should create a consumer and read the message.

The message will be of the type described earlier.

The messages are read-only, which means it cannot be modified after being sent.

The same message can be sent multiple times.

Understanding the Jakarta Messaging - Message

To show a clear example of Jakarta Messaging use cases, we will create a scenario of sending an email.

We will have one producer that will produce the message of an email, and the consumer will get this message and send it to the email.

Both the consumer and the producer will be in the same application, but it can be in separate application as well.

Thus, we will create a new project called `samplejakartamessaging`.

Creating a Producer to Queue

To be Continued...